

## Projeto Full Stack (Parte 2)

# Sistema de Gestão de Condomínios

## Backend (NestJS + MySQL)

### Objetivo

Implementar o backend e integrá-lo ao banco mysql **DBCONDOMINIO** criado na primeira parte do projeto.

## 1. Preparação do Ambiente

### 1.1 Instalar Node.js

Acesse [nodejs.org](https://nodejs.org).

Baixe a versão LTS (recomendada).

Durante a instalação, mantenha as opções padrão.

Verificar se a instalação concluiu com sucesso:

```
node -v  
npm -v
```

### 1.2 Instalar MySQL

Baixe em [dev.mysql.com/downloads](https://dev.mysql.com/downloads).

Instale o MySQL Server e o MySQL Workbench.

Configure usuário root e senha (anote para não esquecer).

Teste a conexão pelo Workbench.

### 1.3 Instalar NestJS CLI

```
npm install -g @nestjs/cli
```

Verifique a instalação do NestJS:

```
nest --version
```

## 2. Criar Projeto NestJS

```
nest new condominio-backend
```

Observação: Estrutura criada ao iniciar o projeto NestJS: src/ com app.module.ts, main.ts, etc.

## 3. Instalar Dependências

### 3.1 MySQL + TypeORM

```
npm install --save @nestjs/typeorm typeorm mysql2
```

### 3.2 Validação de dados

```
npm install --save class-validator class-transformer
```

## 4. Configurar Conexão com MySQL

Edite src/app.module.ts e inclua o código typescript a seguir:

```
import { Module } from '@nestjs/common';  
import { TypeOrmModule } from '@nestjs/typeorm';
```

```
@Module({  
  imports: [  
    TypeOrmModule.forRoot({  
      type: 'mysql',  
      host: 'localhost',  
      port: 3306,  
      username: 'root',  
      password: 'sua_senha',  
      database: 'DBCONDOMINIO',  
      autoLoadEntities: true,  
      synchronize: true, // cria tabelas automaticamente  
    }),  
  ],  
})  
export class AppModule {}
```

## 5. Criar Entidades (Mapeamento das Tabelas)

Exemplo: src/pessoas/pessoa.entity.ts

```
import { Entity, PrimaryGeneratedColumn, Column } from 'typeorm';
```

```
@Entity('PESSOAS')
export class Pessoa {
  @PrimaryGeneratedColumn()
  ID_PESSOA: number;

  @Column({ length: 150 })
  NOME: string;

  @Column({ length: 20 })
  TIPO_PESSOA: string;

  @Column({ unique: true, length: 20 })
  CPF_CNPJ: string;

  @Column()
  DATA_CADASTRO: Date;
}
```

«ATENÇÃO»

**Repita para Moradores, Unidades, Reservas, Boletos, etc. conforme o modelo da Parte 1.**

## 6. Criar Módulos, Services e Controllers

### 6.1 Gerar módulo Pessoas

```
nest generate module pessoas

nest generate service pessoas

nest generate controller pessoas
```

### 6.2 Service (pessoas.service.ts)

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Pessoa } from '../pessoa.entity';

@Injectable()
export class PessoasService {
  constructor(
    @InjectRepository(Pessoa)
    private pessoasRepo: Repository<Pessoa>,
  ) {}
}
```

```

findAll(): Promise<Pessoa[]> {
  return this.pessoasRepo.find();
}

create(pessoa: Pessoa): Promise<Pessoa> {
  return this.pessoasRepo.save(pessoa);
}
}

```

### 6.3 Controller (pessoas.controller.ts)

```

import { Controller, Get, Post, Body } from '@nestjs/common';
import { PessoasService } from './pessoas.service';
import { Pessoa } from './pessoa.entity';

@Controller('pessoas')
export class PessoasController {
  constructor(private readonly pessoasService: PessoasService) {}

  @Get()
  findAll(): Promise<Pessoa[]> {
    return this.pessoasService.findAll();
  }

  @Post()
  create(@Body() pessoa: Pessoa): Promise<Pessoa> {
    return this.pessoasService.create(pessoa);
  }
}

```

### «ATENÇÃO»

**Agora expanda a estrutura para Moradores, Reservas, Boletos, etc..**

## 7. Testar Endpoints

**Inicie o servidor:**

```
npm run start:dev
```

**Via “postman” todos os endpoints:**

Exemplo para Pessoas ...

GET `http://localhost:3000/pessoas`

POST `http://localhost:3000/pessoas` com JSON:

```
{
  "NOME": "João da Silva",
  "TIPO_PESSOA": "FISICA",
  "CPF_CNPJ": "12345678900",
  "DATA_CADASTRO": "2026-03-31"
}
```

## 8. Resultado da parte 2

Backend NestJS conectado ao banco MySQL;

CRUD básico funcionando; e

API REST acessível via navegador ou ferramentas como Postman.